

Extracting NABL application data from applicant documents

How the extraction pipeline works, what we were trying to build, the three approaches we weighed up, and why we shipped the one we did. Written for anyone who needs to understand the system well enough to change it or to argue with it.

1 Overview of this document

OVERVIEW

This report covers the document-extraction half of NABL Document Intelligence: the part that takes the supporting documents an applicant uploads and turns them into a NABL application form somebody only has to check rather than type out. It sets out the goal, what the system has to do and has to be, the three designs we considered, how the one we shipped actually behaves stage by stage, and what we still don't know.

It doesn't cover the eligibility wizard, the review screen's interaction design, or the detail of how the NABL form schema was modelled. Those are written up separately. It also doesn't report accuracy — the measurement work is real and ongoing, but it belongs in its own document rather than folded in here, for the reason set out below.

A NOTE ON WHAT ISN'T IN HERE

We have measured the extraction path against a hand-authored gold set of 51 documents, and we know a good deal about where it does well and where it doesn't. Those figures are deliberately left out of this report, because two things have to be true before any of them can fairly be called the product's number.

First, the configured production model isn't currently reachable: Bedrock refuses every model on the AWS account we're using, with an account-level entitlement error rather than anything we can fix in configuration, so the numbers we have describe a substitute model. Second, our benchmark measures the simple single-call path, not the orchestrated pipeline section 7 describes. The instrument for measuring the real thing exists (`scripts/benchmark_ui_pipeline.py`) and hasn't been run yet. Quoting one design's accuracy as another's would be misleading, so we don't.

2 Goal

GOAL

The goal is to turn a NABL application from a typing exercise into a checking exercise. An applicant uploads the documents they already have — a GST proof, calibration certificates, staff CVs, PT/ILC reports, sometimes a half-finished draft of the form itself — and the system fills in the matching form fields, flagging anything it isn't sure about so a person can confirm it.

What matters is that a reviewer always sits between the model and the submitted form. That changes the accuracy bar completely. The system doesn't have to be right on its own; it has to be right often enough that checking beats typing, and wrong rarely enough that the reviewer keeps trusting what they see. A reviewer who stops trusting the pre-filled values will re-read every document anyway, and then we've built nothing. The project does have a stated accuracy target, and it's tracked in its own document.

Two things follow from that review step. Every extracted value should carry the page it came from and, where the file format allows it, a box around it — so verifying means glancing at the source rather than re-reading the document. And every document should cost a known, bounded amount to process. We added the second goal only after cost became visible; before that, nothing counted how many model calls a document took and nothing stopped it taking more.

3 Requirements

REQUIREMENTS

3a What it has to do

- **Support all nine real NABL application forms** — 151, 152, 153, 153A, 154, 155, 157, 158 and 159. These aren't nine variations on one form; they fall into two genuinely different shapes, and the schema reflects that.
- **Work out what each document is** before trying to read it, across seventeen document types, and accept the formats applicants actually have — PDF, both born-digital and scanned, plus DOCX, JPG and PNG, several at a time.
- **Put values into named form fields** — real NABL paths like `organisation.gst_number`, validated against the schema for whichever form the applicant selected, and tie every value back to the page it came from.
- **Also capture what the schema has no room for** — open key/value pairs under the document's own labels, and results-table rows as rows. A lab report's table is most of its content and none of it fits a NABL form field.
- **Cope with a filled form of fifteen to thirty pages** in one upload, which means reading it in pieces: the text runs well past any context window, and the opening pages are NABL's own boilerplate rather than anything the applicant wrote.

- **Flag uncertainty instead of quietly accepting it.** Open-ended values are assigned a confidence below the review screen's threshold on purpose, so a person confirms them. We never ask the model how confident it is — see the tenets.

3b What it has to be

- **Resilient to a provider having a bad day.** Every call goes through an ordered chain of providers and falls through to the next on any failure. One that keeps failing gets backed off, doubling each time up to a cap, so we stop hammering it.
- **Bounded and honest about cost.** A hard ceiling of six model calls per document, split across classification, extraction and recovery. Vision shares the recovery allowance rather than getting its own, because a vision call is a recovery attempt.
- **Never all-or-nothing.** If the budget runs out or one section fails, the pipeline hands back what it already has and records why it stopped. An upload that got most of the way is far more useful than an error.
- **Able to OCR English without leaving the machine.** English scans go through a local engine, in process. The OCR Lambda is only for devanagari, arabic and the south-Indian scripts it actually has models for.
- **Measured through the same code it ships.** The product and the benchmark share one extraction module — same prompt, same page handling, same output shape, with only the provider chain differing. Without that, the benchmark measures something the product doesn't do, which is exactly the trap we'd fallen into before.

4 Background

BACKGROUND

NABL applications are long, full of tables, and evidence-driven: most fields have to be backed by a certificate the lab already holds. In practice that means somebody sits down and copies serial numbers, calibration dates, staff qualifications and PT scores out of PDFs and into form tables by hand. It's slow, and it's where a lot of the mistakes come from that get an application sent back.

We modelled the schema from the official NABL templates, read in full rather than sampled. Two things from that reading shaped everything afterwards. The first is that the nine forms really do split into two shapes rather than one form with optional sections: the full accreditation applications carry senior management, internal audit, fees and signatories, while the lighter recognition schemes have a flat lab-details section and much narrower tables. Trying to force both into one base would have meant a schema that was wrong about both.

Cost has its own history. Before the budget existed, a one-page certificate cost three model calls — one to classify it, one for schema fields, then a third that read the same page all over again for open fields — and a long filled form could cost upwards of twenty. Nobody knew, because nothing counted. Folding those three reads into one call per chunk, and answering classification locally wherever we can, is what brought the ceiling down to six.

5 Who's your customer

CUSTOMER

The applicant lab's Quality Manager is who we're building for. They're assembling the application, they know their own lab and their standard inside out, and they have no interest in how our software works. They hold all the supporting documents and they're the one currently doing the transcription. They'll judge this on one question: did it save me a day of typing without slipping in a mistake I now have to go and find?

The internal reviewer is who actually operates it — the split-pane screen, confirming pre-filled values against the highlighted source. Every design decision about confidence thresholds and bounding boxes exists for this person. They're also the one who feels a coverage gap most directly: a field the system missed means going back to reading the document themselves, which is the thing we were meant to remove.

The NABL assessor never touches the product, but their acceptance criteria are the real specification. A plausible-looking wrong field costs far more here than a blank one — the blank gets filled in, the wrong one gets submitted. That's why we'd rather the system miss something than guess at it.

6 Tenets

TENETS

These are rules the code already follows. They're written down here so that a future change can be checked against them, and so nobody has to rediscover the reasoning.

Tenet	Why we do it this way
Hitting a ceiling is normal, not an error.	When a document's call budget runs out, the pipeline hands back what it has and records why. Throwing an exception would turn "we stopped early" into "the upload failed", which is a much worse outcome and not what happened.
Never take the model's word for its own confidence.	We deliberately don't ask for a self-reported score. That number is the model's opinion of its own work, and the whole point of the review step is that we don't take that on trust.
A failure has to show up in the score.	When a document fails, it's scored as having predicted nothing, so every expected field counts as missed. A failed document never quietly drops out of the denominator and flatters the result.

Tenet	Why we do it this way
The model reads; it doesn't decide.	Eligibility gating is a plain state machine. The model's only job is turning a free-text answer into a yes or a no. It never decides whether someone qualifies.
Don't invent a number to look precise.	This is open-set extraction, so there's no such thing as a true negative. We say so rather than writing in a zero that would make a textbook accuracy formula look rigorous. A reported token count of zero means the provider didn't tell us, not that the call was free.

7 DESIGN

Design

7a What makes this hard

Four properties of this problem between them rule out the obvious approach of writing down the fields you want and asking a model for them.

- **There are two vocabularies, not one.** The NABL form wants `organisation.gst_number`. A lab report only knows its own labels — `sample_name`, `report_no`. Score one correct read against the wrong vocabulary and you generate a false positive and a false negative out of a single right answer.
- **The field list is open-ended.** There's no fixed set of possible field names, so the question "which fields correctly didn't appear?" has no answer. True negatives don't exist here, which means textbook accuracy can't be calculated at all.
- **Tables have no schema to validate against.** The rows of a results table are the substance of a lab report, and nothing in the NABL form corresponds to them.
- **Pages within one file are not alike.** A twenty-page PDF is routinely native text for two pages, scanned for the next two, then native again. A document-level "does this have a text layer?" check calls the whole file born-digital and silently misreads a third of it.

7b The three designs we weighed up

DESIGN A	One call, whole document kept as our measuring instrument · <code>scripts/generate_predictions_and_score.py</code> A single model call per document. Text or image, no classification, no schema, no grounding — just "tell me every key/value pair and table row in this document, in its own words". It was never a candidate for shipping, but it turned out to be the right instrument for a narrower question: can the model read this kind of document at all?
DESIGN B	Ask per schema section rejected as the only design · <code>extractor.extract_full_form_fields</code> Look at the schema for the chosen form, then make one call per section asking only for that section's named fields. Everything that comes back validates against a real slot, and there's no guessing about names. The problem is that cost scales with how many sections the form has rather than how much of it the document covers — ten calls for NABL 151 whether the document speaks to one section or all of them.
DESIGN C · SHIPPED	Budgeted orchestration over inspected pages <code>documents/orchestrator.py</code> — <code>inspect</code> , <code>classify</code> , <code>retrieve</code> , <code>chunk</code> , <code>extract</code> , <code>validate</code> , <code>recover</code> Seven stages under a single hard call budget, where anything that can be answered without a model is, and the model gets spent only where nothing cheaper will do. It hands back the same field results as the code it replaced, so the compiler, the review screen and the form-fill path never had to know it changed.

How the shipped design actually runs

Stage	Model?	What happens
1 Inspect	No	Page by page, never document by document: what's on each page and how its text was obtained. The text-quality test looks at character count, alphanumeric ratio and word count together rather than trusting a bare character threshold. Tables are detected properly where the PDF library can do it, with a ruled-line heuristic as a fallback so an OCR'd page can still be flagged.
2 Classify	Sometimes	Local scoring runs first, always. We only ask the model when the local answer isn't confident enough and the budget allows it. This is where most of the saving on short documents comes from — every one-page upload used to pay for a call here.
3 Retrieve	No	Field templates get grouped by form section, then the vector store returns only the chunks relevant to each one. That's an embedding lookup per section instead of sending every page to the model for every section.
4 Chunk	No	Relevant pages are grouped into as few chunks as will fit, within a character limit and a maximum number of chunks. When they don't fit, the smallest adjacent pair gets merged — but never at a table boundary while any other boundary is available, so a results table spanning a page break doesn't end up split across two calls that each see half the rows and neither the header.

Stage	Model?	What happens
5 Extract	Yes	One call per chunk that returns everything applicable in a single JSON object: open key/values, table rows, and the named schema slots the form needs. Combining them is safe because the three don't interact, and it's the single biggest saving available. Pattern matching has already taken the identifiers before this runs.
6 Validate	No	Works out cheaply and repeatably which fields are worth spending a recovery call on, and produces two lists: fields that are missing, and fields that are present and well-formed but not supported by the evidence. The old retry pass only looked at empty fields, so a confidently wrong value was never questioned while we spent calls re-asking for things genuinely not in the document.
7 Recover	Yes	Targeted and batched. Gaps in three different sections are one call carrying six fields, not three calls carrying two each, and it never resends the whole document. Escalating to vision is deliberately rare: a low OCR confidence on its own isn't enough, because most low-confidence pages are low-confidence for reasons a second look won't fix.

7c Pros

Design	What's good about it
A · one call	– As cheap as it gets: one call per document. – Separates the model's reading ability from everything the pipeline does, which is what makes it a good measuring instrument. – Cached predictions mean re-scoring costs nothing.
B · per section	– Everything that comes back fits a real schema slot, so it's directly usable. – No naming ambiguity: we supply the field name rather than hoping for it. – Narrow prompts leave the model less room to wander.
C · shipped	– Captures both vocabularies without scoring either against the other, and table rows survive as rows. – Real bounding boxes wherever the format allows, which is what makes checking quick. – Cost is capped and reported, with the reason it stopped recorded. – Four of the seven stages need no model at all, so accuracy work and cost work stop pulling against each other. – Fails in pieces rather than all at once.

7d Cons

Design	What it costs us
A · one call	– Describes nothing the product actually does — no classification, budget or grounding. – Multi-page PDFs go text-only, so a blank or redacted box looks identical to a value that happens to read like the next field's label. – No bounding boxes at all, so its output could never drive the review screen.
B · per section	– Completely blind to anything the schema has no slot for, which loses every table row. – Cost tracks the form's section count, not the document's actual coverage. – A form path can never match a lab report's own vocabulary, so it does worst on precisely the documents applicants upload most often.
C · shipped	– By far the most moving parts, and each stage fails differently. – Open-ended values are unvalidated by construction, so they always land below the threshold and always cost reviewer time. – Adds a vector store to what has to be running. – A wrong local classification is silent — it routes to the wrong field set with no error raised, which is why we hold local confidence to a fairly high bar. – The vision fallback has no reliable per-field box, so those documents drop back to a page-level highlight. – Not yet measured end to end.

We shipped Design C because Design B's blind spot isn't an edge case — table rows are the bulk of what's in the documents applicants actually hold — and because grounding is a hard requirement of the review step, which rules Design A out entirely. Design A survives as the benchmark instrument, which is the job it's genuinely good at.

8 Conclusion

CONCLUSION

The approach works, and more usefully, we now know the shape of what's left rather than guessing at it. The pipeline reads the documents applicants actually upload, keeps results tables intact as tables, ties values back to where they came from, and costs a known amount per document. Four of its seven stages don't touch a model, which is what stopped cost work and accuracy work fighting each other.

Two findings are more tractable than they first look. A meaningful slice of the shortfall is naming rather than reading: the value was pulled out correctly and filed under a different name than the human used, which is a prompt or an alias problem, not a comprehension one. And in results tables, the measured values and reference ranges come out reliably while the test name attached to them is much weaker — the rows are being parsed correctly and then labelled with the wrong analyte. In a lab report that's the most dangerous failure we have, because a correct value under the wrong test name is worse than no value at all, and it's a narrower problem to fix than a general accuracy shortfall.

Two things block calling any of this finished. The production model isn't reachable on our AWS account, so we can't measure what we actually intend to ship on. And the orchestrated pipeline this report describes has no end-to-end measurement of its own yet — what we've measured is the simpler single-call path. Both are prerequisites for a credible accuracy claim, not follow-ups to one.

9 FAQs

FAQS

Open questions this work has raised. They're recorded here without answers on purpose — this is the agenda for the next review, not a summary of settled conclusions.

#	Question
Q1	Nova Lite 2 isn't working, so do we need to re-verify accuracy on it before any of our figures can be treated as the production numbers?
Q2	It failed to read tiny font. How small does text have to get before extraction degrades, and is the fix a higher rasterization DPI, a different OCR model, or sending the page as an image?
Q3	It confused patient name with physician name — but not in every document. What is different about the documents where the two get swapped?
Q4	It failed to capture qualifier values: given “high 5.6 g/mL” it reads the number and drops the “high”. Where should a qualifier live — appended to the value, a separate flag, or its own column on the test row?
Q5	Why do measured results come out so much more reliably than the test names attached to them? Is row order being misaligned against the analyte column, or are names being normalised away?
Q6	Why does the scoring summary report zero model calls after a run that plainly made them, and which earlier runs has that instrumentation gap already misreported?
Q7	Are the renamed fields a prompt problem — pin the vocabulary — or a scoring problem, where we should accept known aliases?
Q8	Is the provider fallback chain really as deep as it's configured to be, or is it effectively only two providers for anything involving an image?
Q9	Medical and cytology documents do markedly worse than milk reports. Should the target be set per domain rather than as one headline number?
Q10	What does the orchestrated pipeline actually score, and how far does it diverge from the single-call path once classification, retrieval and recovery are in play?

10 Appendix

APPENDIX

10a What we measure, and why

We score at two levels, because they answer different questions. The strict level asks whether the system used the same field name a human used, which is what matters if you're filling a named slot on a form. The naming-independent level asks whether the information was extracted at all, under any name, which is a measure of reading ability. Reporting only one of them hides something real: a correct read filed under the wrong name is a genuine cost, but it is a completely different problem from a misread, and it has a different fix.

Two things we deliberately do not report. There is no true-negative count, because in open-set extraction there isn't one; we say so rather than substituting a zero that would make a textbook accuracy formula look authoritative. And ground-truth table rows have no counterpart at all in some extraction paths, so they're reported as an explicit coverage gap rather than counted as misses that would drag the headline down for a reason unrelated to reading accuracy.

10b Operating limits

Setting	Value	Why
Call ceiling, per document	6 total	A hard cap nothing can exceed: 1 classification (skipped when local scoring is confident enough), 4 extraction (one per chunk), 2 recovery (shared with vision escalation, which doesn't get its own).

Setting	Value	Why
Max chunk size	12,000 chars	Tables are never split at a merge boundary
Max images per call	2	Vision escalation is deliberately narrow
Retrieval	top 4, min score 0.25	One vector lookup per form section, no model call
Model max tokens	8,192	Same for both providers
Per-call timeout	180 s	Raised well above the app default; a very long results table needs the headroom
Provider backoff	5 s and 15 s, doubling to 300 s	Rate limits clear faster than a struggling provider recovers
Born-digital page cap	40 pages	A filled form's opening pages are NABL boilerplate, not applicant data
Local classification floor	0.70	Below this we ask the model
Review confidence threshold	0.85	Open-ended values are assigned just under it on purpose
Test suite	261 tests	Model and OCR calls mocked; runs without credentials

10c Edge cases we know about

- **Qualifiers get dropped.** “high 5.6 g/mL” comes back as just the number. The schema has nowhere to put the word.
- **Tiny font.** Small type isn't read reliably, and we haven't yet isolated whether that's rasterization DPI, the OCR model, or where the vision fallback kicks in.
- **Patient name against physician name.** Swapped in some documents, fine in others. Layout proximity is the obvious suspect, but we haven't found what actually distinguishes them.
- **Analyte mislabelling.** Measured values and reference ranges come out well; the test name attached to them is much weaker.
- **A label read as a value.** On a text-only path, a blank or redacted box comes back as the field's own label, because OCR emits no signal at all for “there was a gap here”. Sending the page image alongside the text is what fixed it; the multi-page text-only path still carries the risk.
- **A table spanning a page break.** Never split across two calls, since each would see half the rows and neither the header.
- **Devanagari OCR on English text.** The deployed model garbles English digits and table layout badly enough to misattribute whole rows, which is why English is handled locally instead.
- **Vision fallback loses boxes.** When local OCR can't process an image at all, we fall back to a vision call with no reliable per-field box, and the reviewer gets a page-level highlight rather than a tight one.

10d Glossary

Term	Meaning
NABL	National Accreditation Board for Testing and Calibration Laboratories, the Indian accreditation body whose forms this product fills in.
Open-set extraction	Extraction with no fixed list of possible field names, so true negatives don't exist and textbook accuracy can't be computed.
Grounding	Attaching a source page and bounding box to a value so a reviewer can check it against the document.
HITL	Human in the loop — the split-pane review screen where somebody confirms pre-filled values against the highlighted source.
MU and CMC	Measurement Uncertainty, on testing and medical scopes, and Calibration and Measurement Capability, its calibration-lab counterpart. Kept separate deliberately.
Inference profile	A region-prefixed Bedrock model identifier. Our configured model supports only this style of invocation, never plain on-demand.
Gold set	The hand-authored ground-truth documents we score against, written by hand rather than derived from model output.